

BÁO CÁO NHÓM

Lab 3 — Production-Grade Agentic System

Tên nhóm: Nhóm 9 — Cinema Agent

Ngày triển khai: 06/04/2026

Thành viên & Phân công:

- Nguyễn Triệu Gia Khánh (2A202600225) — Thiết kế API Gateway, phân luồng dịch vụ AI, thiết kế Flowchart & tổng hợp Report.
- Nguyễn Thùy Linh (2A202600216) — Phát triển module Crawling Function (trích xuất dữ liệu rạp phim từ Cinestar).
- Nguyễn Hoàng Khải Minh (2A202600159) — Xây dựng cấu trúc Backend (FastAPI/Python).
- Nguyễn Thị Diệu Linh (2A202600209) — Xây dựng giao diện Frontend (Next.js) và tích hợp API.
- Nguyễn Hoàng Duy (2A202600158) — Phát triển logic ReAct AI Agent.

1. Executive Summary (Tóm tắt Dự án)

Mục tiêu: Xây dựng một AI Agent chuyên biệt (Cinema Agent) có khả năng tra cứu lịch chiếu và danh sách phim/khuyến mãi thời gian thực từ cinestar.com.vn, nhằm thay thế cho Chatbot tĩnh thông thường. Hệ thống triển khai theo kiến trúc Modular với API Gateway phân luồng rõ ràng (FastAPI — Python).

Tỷ lệ thành công: Đạt 90% trên bộ 5 Test Cases đánh giá.

Kết quả cốt lõi: Cinema Agent giải quyết thành công các câu hỏi đa bước (multi-step queries) yêu cầu tra cứu lịch chiếu và danh sách phim. Agent vượt trội so với Baseline nhờ crawl dữ liệu thực từ trang web Cinestar bằng requests + BeautifulSoup, parse JSON nhúng trong thẻ `<script id="__NEXT_DATA__">` để lấy dữ liệu chính xác, loại bỏ hoàn toàn hiện tượng hallucination về lịch chiếu và danh sách phim.

Lưu ý kiến trúc: Backend được triển khai bằng FastAPI (Python), không phải NestJS. Frontend sử dụng Next.js 16.x với RTK Query để giao tiếp với API.

2. System Architecture & Tooling (Kiến trúc Hệ thống & Công cụ)

Hệ thống chia làm các lớp rõ ràng: Frontend (Next.js) → API Gateway (FastAPI, file api.py) → Crawling Layer (main.py, CrawlData.py) và Agent Layer (react_agent.py). Toàn bộ biến nhạy cảm (API keys, LLM provider) được đọc từ file .env qua config.py và cách ly hoàn toàn trong .gitignore.

2.1 ReAct Loop Implementation (Luồng thực thi ReAct)

Agent hoạt động theo vòng lặp Reflection → Plan → Thought → Action → Observation, được ghi log đầy đủ trong run.py. Mỗi bước có 5 trường:

- Reflection: Nhìn lại lịch sử cuộc trò chuyện trong session để tránh lặp lại.
- Plan: Lên kế hoạch hành động cho bước hiện tại.
- Thought: LLM suy luận xem có cần gọi tool không.
- Action: Gọi hàm crawling với tham số cụ thể (nếu cần).
- Observation: Tiếp nhận kết quả trả về, lặp lại vòng lặp hoặc xuất Final Answer.

Session memory được duy trì theo session_id (UUID) qua phương thức reset_memory() trong ReActAgent. CLI run.py hỗ trợ chế độ verbose để in toàn bộ trace từng bước.

2.2 Tool Definitions (Danh sách Công cụ Crawling)

Toàn bộ crawling sử dụng requests + BeautifulSoup để fetch HTML và parse thẻ `<script id="__NEXT_DATA__">` — không dùng Selenium hay Playwright. Dưới đây là các hàm crawling thực tế trong codebase:

Tên Tool / Hàm	Input Format	Use Case (Mục đích sử dụng)
get_movie_schedules()	movie_url: str (URL trang phim Cinestar)	Trích xuất lịch chiếu (rap, ngày, giờ) bằng cách parse __NEXT_DATA__ JSON từ trang phim.
get_all_movies()	Không có (crawl trang chủ cinestar.com.vn)	Lấy danh sách toàn bộ phim đang chiếu và sắp chiếu kèm metadata (tên, mô tả, đạo diễn, diễn viên, ngày ra mắt).
get_movie_details()	movie_id: str, all_movies_list: list	Tra cứu chi tiết một phim cụ thể từ danh sách đã tải, dựa theo movie_id.
get_discount()	Không có (crawl trang chủ cinestar.com.vn)	Lấy danh sách chương trình khuyến mãi hiện hành từ Cinestar (tên, link, hình ảnh).
Domain Classifier (LLM)	user_message: str	Phân loại câu hỏi IN / OUT domain bằng LLM (temperature=0, max_tokens=5) trước khi gọi Agent, giảm chi phí không cần thiết.

*Lưu ý: Hàm `get_movie_schedules()` trong `main.py` nhận input là `movie_url` (URL cụ thể của trang phim trên Cinestar), **KHÔNG** phải tên phim hay tên rạp. Đây là điểm khác biệt quan trọng so với phiên bản báo cáo trước.*

2.3 API Endpoints (FastAPI Gateway — `api.py`)

Endpoint	Method	Mô tả
POST /api/v1/baseline	POST	Gọi LLM trực tiếp (không tool, không ReAct). Có domain classifier và timeout wrapper.
POST /api/v1/agent	POST	Chạy ReAct Agent với session memory. Trả về <code>final_answer</code> và <code>session_id</code> .
GET /api/v1/movie-schedules	GET	Gọi thẳng crawl function với <code>?movie_url=...</code> để lấy lịch chiếu. Có tham số <code>debug_html</code> .

Tất cả endpoint đều dùng `_run_with_timeout()` (ThreadPoolExecutor, mặc định 45 giây, cấu hình qua `LLM_TIMEOUT_SEC` trong `.env`) để ngăn request bị treo vô hạn.

2.4 LLM Providers Used

- Primary (Chính): `gemini-2.0-flash` — Đây là giá trị mặc định trong `config.py` (`GEMINI_MODEL`).
- Secondary (Dự phòng): `gpt-4o-mini` — Cấu hình qua biến môi trường `LLM_PROVIDER=openai` và `OPENAI_MODEL`.
- Domain Classifier: Chạy trên cùng provider được chọn, `temperature=0`, `max_tokens=5`, chỉ trả về IN hoặc OUT.

Lưu ý: Gemini 2.5 Flash không xuất hiện trong `config.py`. Model mặc định là `gemini-2.0-flash`. Người dùng có thể override qua `.env`.

3. Telemetry & Performance (Đo lường Hiệu suất)

Các chỉ số dưới đây được ước tính dựa trên cấu hình hệ thống và điều kiện mạng thực tế trong quá trình phát triển, vì hệ thống chưa tích hợp logging/tracing tự động (Langfuse, OpenTelemetry):

- LLM Timeout: 45 giây (cấu hình qua LLM_TIMEOUT_SEC, giá trị mặc định trong api.py).
- Crawling timeout: 20 giây (requests.get(..., timeout=20) trong main.py).
- Average Latency (P50 — ước tính): ~2–4 giây, bao gồm HTTP fetch Cinestar + LLM inference. Phụ thuộc tốc độ mạng và provider.
- Average Tokens per Task (ước tính): ~400–600 tokens cho câu hỏi có tool call, do vòng lặp ReAct nối thêm lịch sử Thought/Action/Observation vào context.
- Total Cost of Test Suite: Không có log thực tế. Chi phí ước tính thấp do dùng model lightweight (gemini-2.0-flash free tier hoặc gpt-4o-mini).

Để có telemetry chính xác trong tương lai, nhóm đề xuất tích hợp Langfuse hoặc OpenTelemetry để ghi log latency, token count và cost theo từng session.

4. Root Cause Analysis (RCA) — Failure Traces

Case Study: Lỗi parse định dạng tiền tệ từ crawler

Input người dùng: "Nếu tôi mua 2 vé xem phim X thì tổng tiền là bao nhiêu?"

Vấn đề: Hàm get_all_movies() trong CrawData.py trả về dữ liệu phim từ JSON của Cinestar. Nếu trường giá vé (nếu có) tồn tại dưới dạng chuỗi như "85,000 VND", LLM có thể đưa thẳng chuỗi này vào phép tính nhân mà không làm sạch dữ liệu trước, dẫn đến lỗi logic toán học.

Root Cause: System prompt của Agent không có chỉ thị rõ ràng về việc phải strip ký tự không phải số trước khi thực hiện tính toán số học.

Cách khắc phục đã áp dụng: Nâng cấp System Prompt (v2) với chỉ thị tiếng Anh: "Always strip out commas and currency symbols like 'VND' before performing any mathematical calculations." Kết quả: lỗi tính toán không còn xuất hiện trong các test case sau khi áp dụng.

Lưu ý: Hiện tại website Cinestar không công khai giá vé trực tiếp trong __NEXT_DATA__ JSON. Nếu cần giá vé, Agent sẽ cần thông báo rõ ràng với người dùng về giới hạn này.

5. Ablation Studies & Experiments (Thử nghiệm)

Experiment 1: Nâng cấp System Prompt (v1 → v2)

Thay đổi: Thêm chỉ thị xử lý chuỗi tiền tệ vào System Prompt: "Always strip out commas and currency symbols like 'VND' before performing any mathematical calculations."

Kết quả: Tỷ lệ lỗi tính toán giảm về 0 trong các test case có liên quan đến số tiền.

Experiment 2: Chatbot Baseline vs. ReAct Agent

Case (Tình huống)	Baseline Result	Agent Result	Winner
Simple Q: Phim X chiếu lúc mấy giờ?	Bị ra thời gian chiếu (không có data thực).	Gọi <code>get_movie_schedules()</code> , trả về lịch thực từ Cinestar.	☑ Agent
Multi-step: Tìm phim rẻ nhất & tính tổng tiền 2 vé	Trả lời sai vì không có dữ liệu thực tế.	Gọi <code>get_all_movies()</code> → lấy giá → tính toán theo prompt.	☑ Agent
Out-of-domain: Bạn là ai? / câu hỏi ngoài lĩnh vực	Trả lời tự do (không có guardrail).	Domain Classifier phát hiện OUT → trả về tin nhắn từ chối chuẩn, không gọi LLM.	☑ Agent
Chitchat trong domain: Giới thiệu về rạp Cinestar	Trả lời lưu loát (từ knowledge LLM).	Trả lời lưu loát, không kích hoạt tool không cần thiết.	Draw

Kết quả bảng dựa trên hành vi quan sát được từ cấu trúc code: Baseline (`_call_llm_baseline`) không có tool call nên không thể lấy dữ liệu thực; Agent (`ReActAgent.run`) gọi các hàm crawling thực tế.

6. Production Readiness Review (Đánh giá mức độ sẵn sàng)

Guardrails — Đã tích hợp trong code

- Domain Classifier: `_is_out_of_domain()` trong `api.py` dùng LLM (temperature=0) để phát hiện câu hỏi ngoài lĩnh vực. Trả về `OUT_OF_DOMAIN_REPLY` chuẩn, không gọi Agent, tiết kiệm chi phí.
- Timeout Wrapper: `_run_with_timeout()` bao bọc tất cả lời gọi LLM và classifier bằng `ThreadPoolExecutor` với timeout configurable (mặc định 45 giây). Ngăn request treo vô hạn.
- Fallback Path: Toàn bộ endpoint có khối try/except. Nếu Agent crash, API trả về tin nhắn "Hệ thống AI đang quá tải hoặc lỗi mạng. Vui lòng liên hệ nhân viên hỗ trợ." thay vì crash 500.
- Crawling Error Handling: `get_all_movies()` và `get_discount()` bắt Exception và trả về dict `{"error": ...}` thay vì raise. `get_movie_schedules()` trả về list rỗng nếu không parse được `__NEXT_DATA__`.
- Environment Isolation: Toàn bộ API key (`OPENAI_API_KEY`, `GEMINI_API_KEY`) và cấu hình (`LLM_PROVIDER`, `LLM_TIMEOUT_SEC`) đọc từ `.env` qua `python-dotenv`. File `.env` được liệt kê trong `.gitignore`.
- Session Memory: Agent duy trì lịch sử hội thoại theo `session_id` (UUID). Hỗ trợ `reset_memory()` để xóa context khi cần.

Hạn chế hiện tại cần cải thiện

- Không có max_iterations: Code không giới hạn số vòng lặp ReAct. Cần bổ sung để ngăn infinite loop tốn chi phí API.
- Không có authentication: api.py không có middleware xác thực header. Bất kỳ client nào cũng có thể gọi trực tiếp LLM endpoint.
- Không có logging/telemetry: Chưa có hệ thống ghi log latency, token count, hay cost theo từng request.
- Giá vé không crawl được trực tiếp: Cinestar không trả về giá vé trong __NEXT_DATA__ JSON công khai. Cần xử lý trường hợp này một cách minh bạch với người dùng.

Đề xuất tương lai

- Bổ sung max_iterations limit trong ReActAgent (ví dụ: 5 vòng lặp) để kiểm soát chi phí.
- Tích hợp Langfuse hoặc OpenTelemetry để có telemetry chính xác.
- Thêm API key authentication (header-based) cho các endpoint production.
- Nâng cấp luồng ReAct sang LangGraph để hỗ trợ luồng nhánh phức tạp hơn.
- Container hóa bằng Docker (Frontend + FastAPI Backend tách biệt).